

Expense Tracking Application - Project Workflow

Project Overview

The Expense Tracking Application is a full-stack web application built with React.js and Java Spring Boot. It allows users to register, log in, manage income and expenses, organize transactions by category, view dashboard summaries, filter records, and generate reports.

High-Level Architecture Workflow

User

- > React Frontend
- > Spring Boot REST API
- > Service Layer
- > Repository Layer
- > H2 Database

Response Flow

H2 Database -> Repository -> Service -> REST Controller -> React UI -> User

Main Modules

- Authentication
- Dashboard
- Income and Expense Management
- Category Management
- Search and Filters
- Reports and Analytics
- CSV Export and Dark Mode

Authentication and Dashboard Workflow

Authentication Workflow

1. User opens the React application.
2. User chooses Login or Register.
3. React sends credentials to the Spring Boot authentication API.
4. Backend validates registration fields or checks login password.
5. Backend returns an authentication token and user details.
6. React stores the session in browser local storage.
7. Protected API calls include the token in the X-Auth-Token header.
8. User can log out, which clears the token.

Dashboard Workflow

1. User logs in successfully.
2. React calls the dashboard API.
3. Backend loads the user's transactions.
4. Service layer calculates total income, expenses, balance, monthly income, monthly expenses, and recent transactions.
5. React displays summary cards, recent transactions, and charts.

Dashboard Outputs

- Total Income
- Total Expenses
- Current Balance
- Monthly Summary
- Recent Transactions

Category and Transaction Workflow

Category Management Workflow

1. User opens the Categories page.
2. React fetches all categories for the logged-in user.
3. User creates, edits, or deletes a category.
4. Backend validates category name and category type.
5. Backend saves changes through JPA repositories.
6. React refreshes the category list.
7. Categories used by transactions are protected from deletion.

Transaction Management Workflow

1. User opens the Transactions page.
2. React fetches categories and transaction records.
3. User adds income or expense with amount, category, date, description, and recurring flag.
4. Backend validates amount, date, category ownership, and category type.
5. Backend stores the transaction in the database.
6. User can edit or delete existing transactions.
7. Dashboard and reports update after changes.

Transaction Types

- INCOME
- EXPENSE

Search, Filter, Reports, and Analytics Workflow

Search and Filter Workflow

1. User enters a search term or selects filters.
2. Filters can include date range, category, and transaction type.
3. React sends query parameters to the transactions API.
4. Backend applies filters in the repository query.
5. React displays the matching transactions.

Reports and Analytics Workflow

1. User opens the Reports page.
2. User selects a date range.
3. React calls the reports API.
4. Backend groups transaction data into monthly expenses, category-wise expenses, and income vs expense totals.
5. React renders pie and bar charts.
6. User can export report data to CSV.

Report Types

- Monthly Expense Report
- Category-wise Expense Report
- Income vs Expense Report
- Date Range Report

Database and Deployment Workflow

Database Tables

Users: stores user profile, email, password hash, and created date.

Categories: stores user-specific income and expense categories.

Transactions: stores category relationship, amount, type, date, description, and recurring status.

Relationships

- One user has many categories.
- One user has many transactions.
- One category has many transactions.

Local Run Workflow

1. Start backend with: `mvn spring-boot:run`
2. Backend runs at: `http://localhost:8080`
3. Start frontend with: `npm run dev`
4. Frontend runs at: `http://localhost:5173`
5. Login using demo credentials or create a new account.

Deployment Workflow

1. Build backend using: `mvn clean package`
2. Build frontend using: `npm run build`
3. Host backend JAR on a server with Java installed.
4. Host frontend dist folder on a static server.
5. Configure frontend API URL for the deployed backend.
6. Replace H2 with MySQL, PostgreSQL, or SQL Server for production use.